ASSIGNMENT 9.1

Hemavathi N

2303A51965

Batch 24

AIAC

Problem 1:

Consider the following Python function:

def find_max(numbers):

return max(numbers)

Task:

• Write documentation for the function in all three formats:

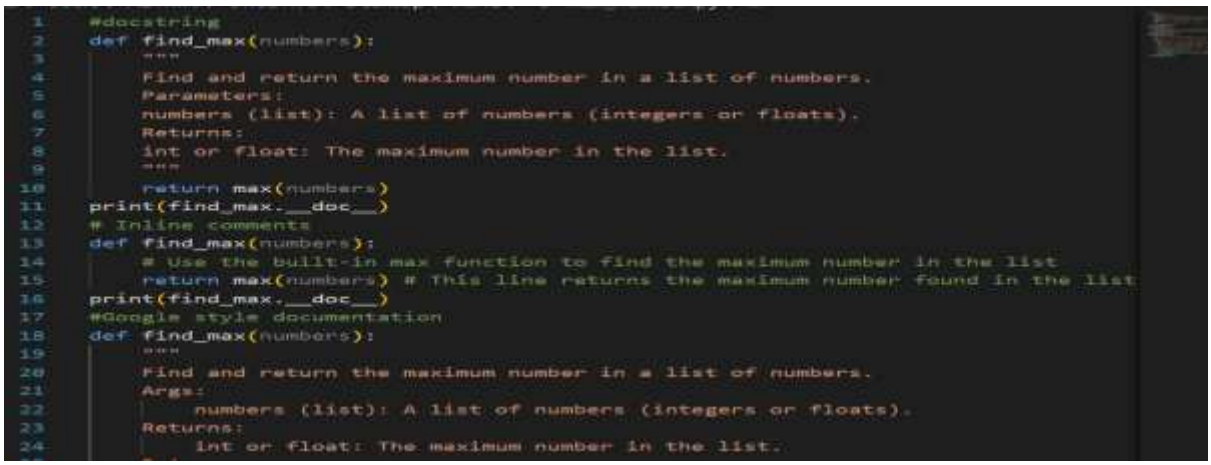
(a) Docstring

(b) Inline comments

(c) Google-style documentation

• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.

• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.



```
#docstring
def find_max(numbers):
    """
    Find and return the maximum number in a list of numbers.
    Parameters:
    numbers (list): A list of numbers (integers or floats).
    Returns:
    int or float: The maximum number in the list.
    """
    return max(numbers)
print(find_max.__doc__)
# Inline comments
def find_max(numbers):
    # Use the built-in max function to find the maximum number in the list
    return max(numbers) # This line returns the maximum number found in the list
print(find_max.__doc__)
#Google style documentation
def find_max(numbers):
    """
    Find and return the maximum number in a list of numbers.
    Args:
        numbers (list): A list of numbers (integers or floats).
    Returns:
        int or float: The maximum number in the list.
```

Output:

```
Find and return the maximum number in a list of numbers.
Parameters:
numbers (list): A list of numbers (integers or floats).
Returns:
int or float: The maximum number in the list.

None

Find and return the maximum number in a list of numbers.
Args:
    numbers (list): A list of numbers (integers or floats).
Returns:
    int or float: The maximum number in the list.
Raises:
    ValueError: If the input list is empty.
    TypeError: If the input is not a list of numbers.
```

Problem 2: Consider the following Python function:

def login(user, password, credentials):

return credentials.get(user) == password

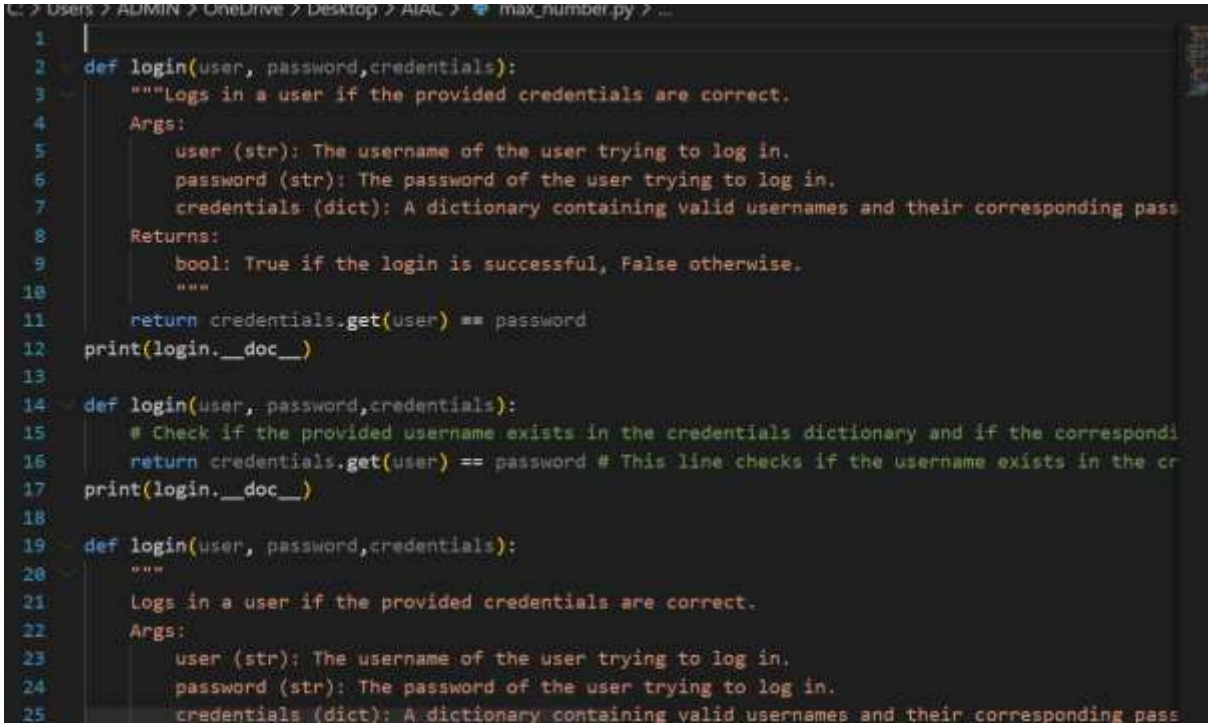
Task:

1. Write documentation in all three formats.

2. Critically compare the approaches.

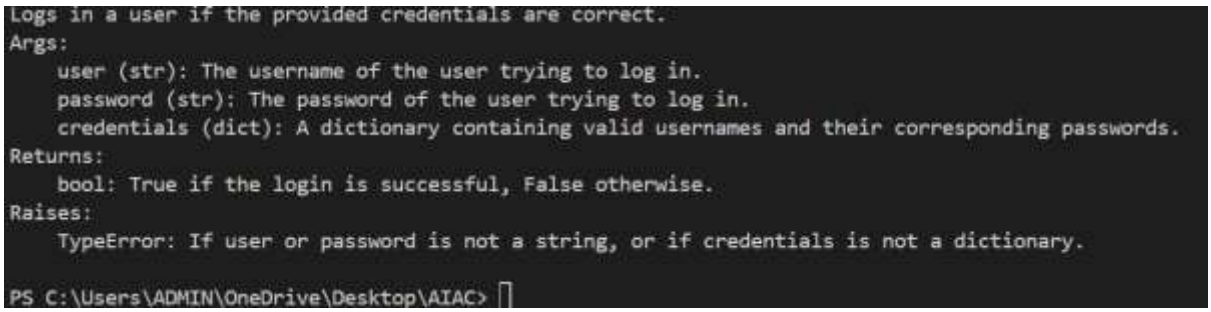
3. Recommend which style would be most helpful for new

developers onboarding a project, and justify your choice.



```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > max_number.py > ...
1
2   def login(user, password,credentials):
3       """Logs in a user if the provided credentials are correct.
4       Args:
5           user (str): The username of the user trying to log in.
6           password (str): The password of the user trying to log in.
7           credentials (dict): A dictionary containing valid usernames and their corresponding pass
8       Returns:
9           bool: True if the login is successful, False otherwise.
10          """
11      return credentials.get(user) == password
12  print(login.__doc__)
13
14  def login(user, password,credentials):
15      # Check if the provided username exists in the credentials dictionary and if the correspondi
16      return credentials.get(user) == password # This line checks if the username exists in the cr
17  print(login.__doc__)
18
19  def login(user, password,credentials):
20      """
21      Logs in a user if the provided credentials are correct.
22      Args:
23          user (str): The username of the user trying to log in.
24          password (str): The password of the user trying to log in.
25          credentials (dict): A dictionary containing valid usernames and their corresponding pass
```

Output :



```
Logs in a user if the provided credentials are correct.
Args:
    user (str): The username of the user trying to log in.
    password (str): The password of the user trying to log in.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
Returns:
    bool: True if the login is successful, False otherwise.
Raises:
    TypeError: If user or password is not a string, or if credentials is not a dictionary.

PS C:\Users\ADMIN\OneDrive\Desktop\AIAC>
```

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and

demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the

following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

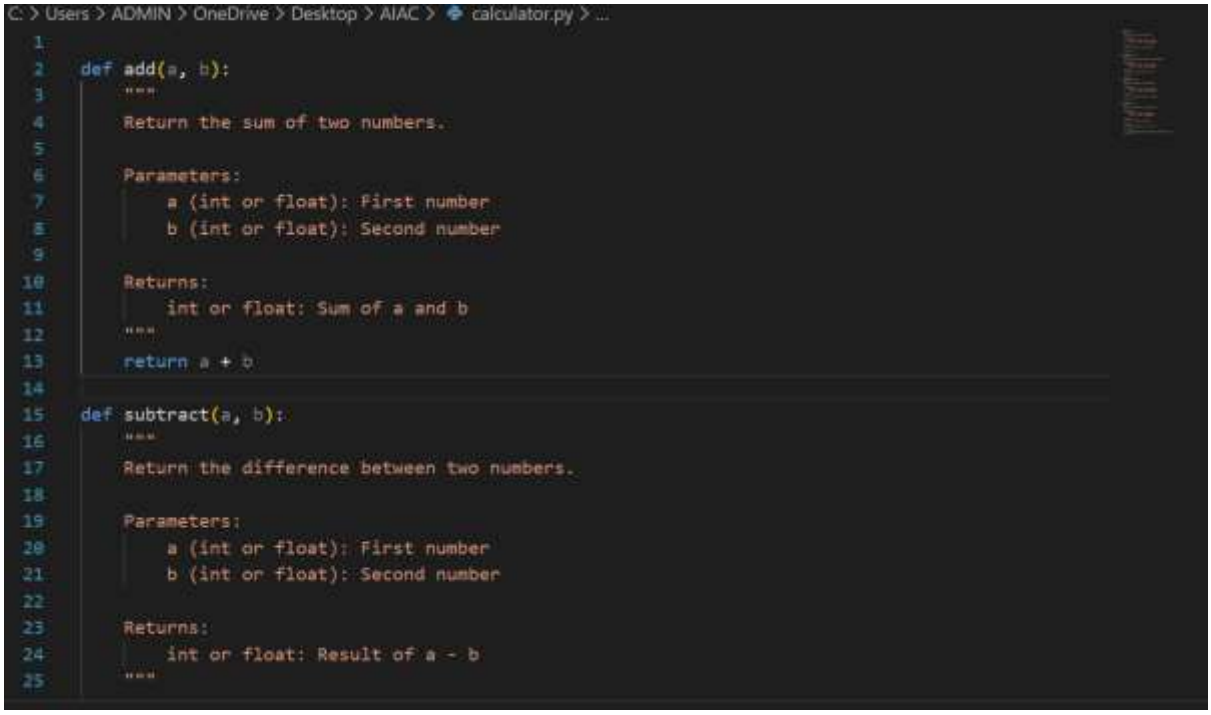
o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

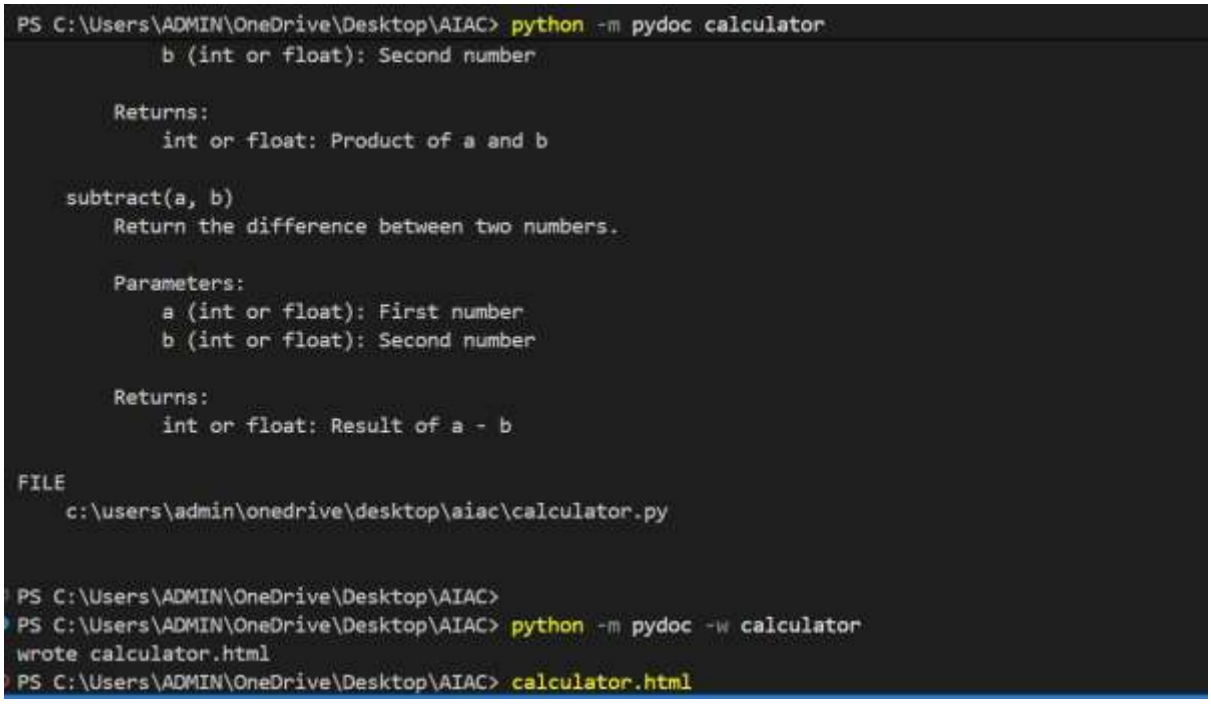
2. Display the module documentation in the terminal using

Python’s documentation tools.

3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.



Output:



index
calculator c:\users\admin\onedrive\desktop\aiac\calculator.py

Functions

add(a, b)
Return the sum of two numbers.

Parameters:
a (int or float): First number
b (int or float): Second number

Returns:
int or float: Sum of a and b

divide(a, b)
Return the quotient of two numbers.

Parameters:
a (int or float): Numerator
b (int or float): Denominator

Returns:
float: Result of division

Raises:
ZeroDivisionError: If b is zero

multiply(a, b)
Return the product of two numbers.

Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

browser.

C: > Users > ADMIN > OneDrive > Desktop > AIAC > convension.py > decimal_to_binary

```python
def decimal_to_binary(n):
    """
    Convert a decimal integer to its binary representation.

    Parameters:
        n (int): A non-negative decimal number.

    Returns:
        str: Binary representation of the number.

    Raises:
        ValueError: If the input is negative.
        TypeError: If input is not an integer.
    """
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")

    if n < 0:
        raise ValueError("Input must be non-negative")

    return bin(n)[2:]
```

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> python -m pydoc convension
Help on module convension:

NAME
    convension

FUNCTIONS
    binary_to_decimal(b)
        Convert a binary string to its decimal representation.

        Parameters:
            b (str): A string containing a binary number.

        Returns:
            int: Decimal equivalent of the binary number.
```

index
convension c:\users\admin\onedrive\desktop\aiac\convension.py

Functions

binary_to_decimal(b)
Convert a binary string to its decimal representation.

Parameters:
b (str): A string containing a binary number.

Returns:
int: Decimal equivalent of the binary number.

Raises:
ValueError: If the string is not a valid binary number.

decimal_to_binary(n)
Convert a decimal integer to its binary representation.

Parameters:
n (int): A non-negative decimal number.

Returns:
str: Binary representation of the number.

Raises:
ValueError: If the input is negative.
TypeError: If input is not an integer.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

browser.

```python
courses = {}
def add_course(course_id, name, credits):
    """
    Add a new course to the course collection.
    Parameters:
        course_id (str): Unique identifier for the course.
        name (str): Name of the course.
        credits (int): Number of credits for the course.
    Returns:
        dict: The course that was added.
    Raises:
        ValueError: If the course already exists.
    """
    if course_id in courses:
        raise ValueError("Course already exists")
    courses[course_id] = {
        "name": name,
        "credits": credits
    }
    return courses[course_id]

```

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> python -m pydoc course
        Raises:
            KeyError: If the course does not exist.

DATA
    courses = {}

FILE
    c:\users\admin\onedrive\desktop\aiac\course.py


PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> python -m pydoc -w course
wrote course.html
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> start course.html
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC>
```

index

course c:\users\admin\onedrive\desktop\aiac\course.py

Functions

add_course(course_id, name, credits)

Add a new course to the course collection.

Parameters:

course_id (str): Unique identifier for the course.

name (str): Name of the course.

credits (int): Number of credits for the course.

Returns:

dict: The course that was added.

Raises:

ValueError: If the course already exists.

get_course(course_id)

Retrieve details of a course.

Parameters:

course_id (str): Unique identifier for the course.

Returns:

dict: Course details including name and credits.

Raises:

KeyError: If the course does not exist.

remove_course(course_id)

Remove a course from the course collection.

Parameters:

course_id (str): Unique identifier for the course.

Returns:

dict: The removed course details